

Lesson 1 — What is Machine Learning?

Let's start from zero and build your ML knowledge step by step. Since you're learning Scikit-learn, I'll explain the concepts first and then show you the code in your Jupyter Notebook.

1. What is Machine Learning?

In traditional programming, we write the rules.

For example, suppose we want to determine whether a person will get a loan:

```
Income > ₹50,000  
AND  
Credit Score > 700  
↓  
Approve
```

We manually create the rules.

Machine Learning is different

Instead of manually writing all the rules, we give the computer examples.

Income	Credit Score	Loan Decision
₹80,000	750	Yes
₹30,000	600	No
₹90,000	780	Yes
₹25,000	550	No
...		

The ML algorithm looks at these examples and learns a pattern.

Then we can give it a new person:

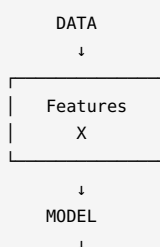
```
Income = ₹70,000  
Credit Score = 720
```

and the model predicts:

```
Loan → Yes
```

2. The Basic ML Workflow

Almost every supervised ML problem follows this basic idea:



Target
y

But the actual learning process is:

```
Training Data
↓
X + y
↓
ML Model
↓
Model learns patterns
↓
Trained Model
↓
New X
↓
Prediction
```

3. What are **x** and **y**?

This is very important, and you were already asking about this.

x = Features / Inputs

These are the pieces of information we give to the model.

For house-price prediction:

```
X
|
├─ Area
├─ Bedrooms
├─ Bathrooms
├─ Age
└─ Location
```

y = Target / Output

This is what we're trying to predict.

```
y
|
└─ House Price
```

So:

```
X
↓
ML Model
↓
y
```

For example:

```
Area = 1500 sq ft
Bedrooms = 3
Bathrooms = 2
Age = 5 years
↓
MODEL
```

↓
Price = ₹80 lakh

4. Regression vs Classification

There are two major types of supervised learning you should understand first.

Regression

When the output is a continuous numerical value.

Examples:

House price → ₹85,00,000
Salary → ₹12,50,000
Temperature → 31.5°C
Sales → ₹2,35,000

So:

X → Model → Number

Your house-price problem is regression.

Classification

When the output is a category/class.

Examples:

Spam / Not Spam
Fraud / Not Fraud
Disease / No Disease
Pass / Fail
Churn / No Churn

So:

X → Model → Class

5. Our First Dataset

Let's use the California Housing dataset because `load_boston()` was removed from modern Scikit-learn.

In your Jupyter Notebook, run:

```
from sklearn.datasets import fetch_california_housing  
  
X, y = fetch_california_housing(return_X_y=True)
```

Now inspect them:

X.shape

and:

```
y.shape
```

You'll see approximately:

```
x → (20640, 8)
y → (20640,)
```

That means:

20,640 houses/examples

and

8 features for each house.

6. Look at x

Run:

```
x[:5]
```

You'll see something like:

```
[[8.3252, 41.0, 6.9841, ...]
 [8.3014, 21.0, 6.2381, ...]
 [7.2574, 52.0, 8.2881, ...]
 ...]
```

Each row represents one house/district example.

Each column represents a feature.

You can see the feature names:

```
housing = fetch_california_housing()

housing.feature_names
```

You'll get:

```
['MedInc',
 'HouseAge',
 'AveRooms',
 'AveBedrms',
 'Population',
 'AveOccup',
 'Latitude',
 'Longitude']
```

7. Look at y

Run:

```
y[:5]
```

You'll see values such as:

```
[4.526, 3.585, 3.521, 3.413, 3.422]
```

These are the target values — the median house value for the corresponding examples.

So conceptually:

```
X[0] → information about house/district #1
y[0] → price/value of house/district #1

X[1] → information about #2
y[1] → price/value of #2
```

This relationship is critical:

```
X[0] → y[0]
X[1] → y[1]
X[2] → y[2]
...
```

8. How Does the Model Learn?

Suppose we have:

```
X = House information
y = House price
```

We give both to the model:

```
model.fit(X, y)
```

The `.fit()` function means:

Train the model using X and y.

The model tries to find relationships between the features and target.

Then:

```
model.predict(X)
```

means:

Use the trained model to make predictions for X.

9. Your KNN Example

You previously had:

```
from sklearn.neighbors import KNeighborsRegressor

mod = KNeighborsRegressor()

mod.fit(X, y)

mod.predict(X)
```

Now you should understand what each line means:

Line 1

```
from sklearn.neighbors import KNeighborsRegressor
```

Import the KNN regression algorithm.

Line 2

```
mod = KNeighborsRegressor()
```

Create the model.

Line 3

```
mod.fit(X, y)
```

Give the model:

```
X = features  
y = answers
```

so it can learn from them.

Line 4

```
mod.predict(X)
```

Ask the trained model:

“What do you predict for these inputs?”

10. One Important Problem

You shouldn't normally do:

```
model.fit(X, y)  
model.predict(X)
```

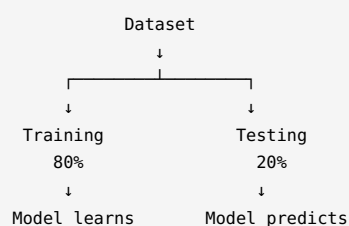
for evaluating your model.

Why?

Because the model has already seen the data.

It's like giving you an exam, showing you all the answers beforehand, and then asking you to take the same exam.

Instead, we split the data:



In Scikit-learn:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Then:

```
model.fit(X_train, y_train)
```

and:

```
predictions = model.predict(X_test)
```

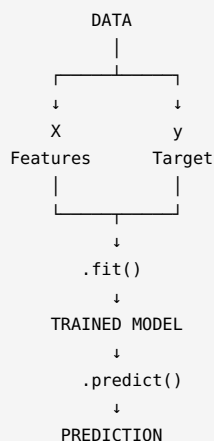
Now we're testing the model on data it didn't train on.

🔗 Lesson 1 — What You Should Remember

Don't memorize algorithms yet. Understand these six things:

1. ML learns patterns from data.
2. X = features / inputs.
3. y = target / output.
4. Regression → predicts numbers.
5. Classification → predicts categories.
6. `fit()` → train the model.
`predict()` → make predictions.

And the most important picture:



Your first exercise

In your notebook, run these one by one:

```
from sklearn.datasets import fetch_california_housing

X, y = fetch_california_housing(return_X_y=True)

print(X.shape)
print(y.shape)

print(X[:5])
print(y[:5])
```

Then:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
```

Once you understand why `X_train`, `X_test`, `y_train`, and `y_test` exist, we'll move to Lesson .